



PARALLEL AND DISTRIBUTED COMPUTING

6 – esercitazione

Marco Lapegna
Dipartimento di Matematica e Applicazioni
Università degli Studi di Napoli Federico II

www.docenti.unina.it/marco.lapegna

1

Marco Lapegna
Parallel and Distributed Computing
6 - esercitazione

esercizio

Data una matrice quadrata di ordine N , scrivere una function con NT thread che calcoli il

$$R = \max_{i=1..N} \sum_{j=1}^N \sqrt{A(i,j)}$$

Eeguire con $N = 800, 1600, 2400, 3200, 4000, 4800, 5400$

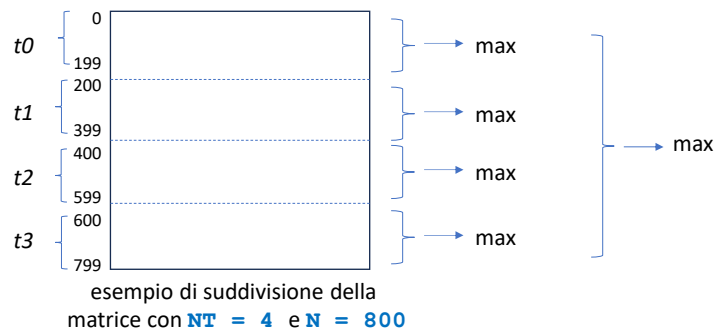
Eeguire con $NT = 1, 2, 4, 8$

Calcolare Speed-UP e Efficienza

$$S_P = \frac{T(N,1)}{T(N,P)} \quad E_P = \frac{S_P}{P}$$

2

- si assegna ad ogni thread una porzione della matrice
- ogni thread calcola il massimo delle righe a lui assegnata
- infine si calcola il massimo tra i valori ottenuti



3

```
double maxsum( int N, int LD, double *A, int NT)
```

valori di input

- int N ordine della matrice
- int LD Leading Dimension dell'array
- double *A puntatore alla matrice (al primo elemento)
- int NT numero di thread

valore di output

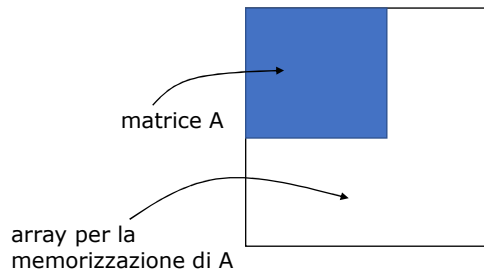
- double in valore di $R = \max_{i=1..N} \sum_{j=1}^N \sqrt{A(i,j)}$

check: Nel caso di una matrice con $N = 800$, inizializzata (riga per riga) con la funzione `rand( )%100` (numeri interi compresi tra 0 e 99 inclusi) la soluzione e' **5471.767610**

4

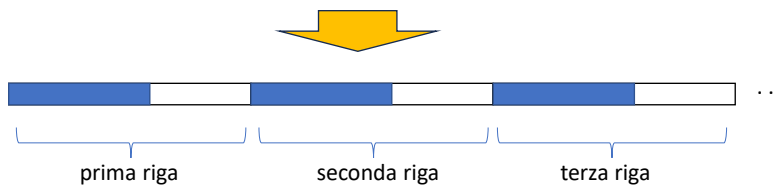
un ripasso sulla gestione delle matrici nelle function

In generale, e' importante tenere ben distinti le matrici e gli array dove esse sono memorizzate (le dimensioni possono essere differenti!)



Inoltre:

- array memorizzati in locazioni contigue
- array 2-dimensionali memorizzati per righe
- passaggio alla function del solo indirizzo del primo elemento (prog. chiamante e function condividono le locazioni di memoria)



Se l'ordine della matrice e' differente dalla dimensione dell'array, allora gli elementi della matrice non sono memorizzati in maniera contigua !!

5

esempio senza Leading Dimension

programma chiamante

```
A = (double*)malloc(sizeof(double)*3*3);
N = 2;

matmat(A, ... N, ...);
```

function

```
void matmat (double *A, ... int N, ...){
}
```

indici nel prog. chiam.	memoria	indici nella function
A(0,0)		A(0,0)
A(0,1)		A(0,1)
A(0,2)		A(1,0)
A(1,0)		A(1,1)
A(1,1)		
A(1,2)		
A(2,0)		
A(2,1)		
A(2,2)		

Senza la Leading Dimension nella function non e' possibile determinare correttamente la posizione di A(i,j)

6

esempio con Leading Dimension

programma chiamante

```
N = 2; ld = 3;
A = (double*) malloc(sizeof(double)*ld*ld);

matmat(ld,... A, ... N, ...);
```

function

```
void matmat (int ld,... double *A, ... int N, ...) {
}

```

indici nel prog. chiam.	memoria	indici nella function
A(0,0)		A(0,0)
A(0,1)		A(0,1)
A(0,2)		A(0,2)
A(1,0)		A(1,0)
A(1,1)		A(1,1)
A(1,2)		A(1,2)
A(2,0)		A(2,0)
A(2,1)		A(2,1)
A(2,2)		A(2,2)

nella function, la posizione di A(i,j) in memoria e' $A + i*ld + j$

7

misurare il tempo di esecuzione

per misurare il tempo di esecuzione, utilizzare la funzione `get_cur_time( )` presente nel file `c_timer.c` che ritorna il tempo corrente del sistema in secondi. La differenza tra due successive chiamate fornisce il tempo di esecuzione di una sezione di codice. La precisione è accurata al microsecondo (10^{-6} sec)

```
#include <stdio.h>
#include <omp.h>
int main() {

    double get_cur_time()
    double t1, t2;
    ...
    t1 = get_cur_time( );
    ...
    R = maxsum(N, LD, A, NT);
    ...
    t2 = get_cur_time( );
    printf( " il tempo di esecuzione e' %f \n ", t2-t1 );
}
```

8

speed-up ed efficienza

Speed-up	NT=1	NT=2	NT=4	NT=8
N=800	1	1.15	1.49	1.12
N=1600	1	1.72	2.43	3.49
N=2400	1	1.64	2.81	4.75
N=3200	1	1.81	3.08	5.85
N=4000	1	1.88	3.47	6.25
N=4800	1	1.93	3.54	6.69
N=5400	1	1.92	3.55	7.07

Efficienza	NT=1	NT=2	NT=4	NT=8
N=800	1	0.57	0.37	0.14
N=1600	1	0.86	0.60	0.43
N=2400	1	0.82	0.70	0.59
N=3200	1	0.90	0.77	0.73
N=4000	1	0.94	0.86	0.78
N=4800	1	0.96	0.88	0.83
N=5400	1	0.96	0.88	0.88

9

consegna

- inserire la function maxsum in un file maxsum.c (N.B. SOLO la function: no main.c, no c_timer.c)
- inserire il file maxsum.c nella propria cartella Dropbox (N.B. solo il file maxsum.c)
- e' possibile lavorare in coppie (N.B. coppia = 2 studenti)
 - entrambi gli studenti inseriscono lo stesso file nella propria cartella Dropbox
 - la prima riga del file deve essere un commento con i nomi degli studenti

10